The Open Source Audit

Subject: **Linux Kernel**

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Units 1–5

Student Name	Apurv Gupta
Registration Number	24BHI10011
Course	Open Source Software
Chosen Software	Linux Kernel
License	GNU General Public License v2 (GPL v2)
Date of Submission	

“What I am doing is making a free version of a minix-lookalike for AT-386 computers.”

: Linus Torvalds, August 25, 1991

Table of Contents

1. Introduction

The Linux Kernel is arguably the most consequential piece of software ever written, not because it is the most technically complex, though it is extraordinarily so, but because of what it represents: a single student's frustration transformed into the backbone of the modern digital world. Today, the kernel runs on everything from Android smartphones in rural India to the supercomputers that simulate climate change, from the servers that power Google and Amazon to the embedded systems inside cars and televisions.

This report audits the Linux Kernel as an open-source project. It examines the historical and philosophical context of its creation, the legal framework of its GPL v2 license, the ethical questions that open-source development raises, its practical footprint on a Linux system, the vast ecosystem it has spawned, and an honest comparison against proprietary operating system kernels. The goal is not to celebrate Linux uncritically, but to understand it, its origins, its freedoms, its limitations, and its place in the broader story of how software shapes society.

2. Part A : Origin and Philosophy

A1. The Problem the Linux Kernel Was Created to Solve

In 1991, your options were basically: pay for Unix, or go without. Unix had been born at AT&T Bell Labs in the late 1960s and spent the next two decades becoming the gold standard for serious computing. Powerful, elegant, and completely proprietary. Universities could license educational versions, but the code was locked. You could sometimes read it, but modifying or redistributing it freely wasn't on the table.

Stallman had been angry about this since 1983. A broken printer at MIT, a manufacturer who wouldn't share the source code so anyone could fix it, and that was enough. He launched the GNU Project with one goal: build a complete Unix-like operating system that was free, meaning freedom, not price. By 1991 it had produced compilers, editors, shell utilities, libraries, pretty much everything you'd need. Except a kernel. That's the core piece that actually talks to the hardware and manages memory and processes, and GNU's answer to it, the Hurd, was still years away from working.

Linus Torvalds was 21, a computer science student in Helsinki, messing around with a 386 PC. He'd been using MINIX, Andrew Tanenbaum's small educational Unix clone, but MINIX was restricted too. Its microkernel architecture, which Tanenbaum genuinely believed was the future of OS design, made it difficult to modify the way Torvalds wanted. His needs weren't even that grand, terminal emulation, remote access to his university's Unix servers. MINIX couldn't give him that cleanly, so he started building something himself.

On August 25, 1991, Torvalds posted a message to the comp.os.minix newsgroup that people still quote today: “I’m doing a (free) operating system (just a hobby, won’t be big and professional like gnu) for 386(486) AT clones.” He was underselling it, to put it mildly. He named it Linux, a mix of his own name and Unix.

Sharing the code wasn’t some ideological stand. That’s just what you did in the early internet hacker community: you built something, you posted it, people gave feedback, some of them sent patches. The early Linux license wasn’t even free software by Stallman’s standards since it banned commercial use. In 1992, Torvalds relicensed it under the GNU GPL v2, and later called it “the best thing I ever did.” It’s hard to argue with that. The GPL meant anyone distributing Linux or a modified version had to release their source code under the same terms. The commons couldn’t be quietly bought up and closed off.

So what did Linux actually solve? Two things, really. On the technical side, it gave the GNU Project the kernel it was missing, turning a pile of excellent but incomplete free software into an operating system you could actually run. The social side is almost more interesting: it was the first real proof that loosely organized volunteers, coordinating over the internet with no central authority, could build something that didn’t just match commercial software but eventually beat it. By the mid-1990s Linux was on web servers. By the 2000s it owned the internet’s infrastructure. One student’s weekend project became the thing the digital economy runs on.

A2. The License: What GPL v2 Actually Says

The Linux kernel is licensed under the GNU General Public License version 2, the GPL v2. It's probably the most consequential legal document in software history, which sounds like an exaggeration until you think about how much of the modern internet runs on this kernel.
To get why the license matters, you need to know the four freedoms Stallman built it around. The freedom to run the software for any purpose. The freedom to study how it works and change it, which only means anything if you can actually read the source code. The freedom to share copies with others. And the freedom to distribute your modified versions, so improvements don't just die with you.
GPL v2 gives you all of that. Download the source, read it, modify it, run it on any hardware, redistribute it. But there's a catch, and it's the whole point: if you distribute a modified version, your changes have to be released under GPL v2 as well. You can't take the code, improve it, and then lock your version down. Because of copyleft. using the same legal tool that usually protects proprietary software to instead guarantee that Linux and anything derived from it stays open. The commons can't be enclosed.

Can a company take Linux, modify it, and sell it? Yes and no, depending on what "sell" actually means. Red Hat, SUSE, and Canonical have been charging money for Linux-based products for decades, so commercial use isn't the issue. The issue is distribution. If you ship hardware with Linux on it, or hand binaries to customers, you have to make your kernel source code available to those customers. You can't pocket the improvements. What you *can* keep proprietary is the application software sitting on top of the kernel, as long as it isn't statically linked into it. That boundary, where the kernel ends and userspace begins, has been argued about in legal circles for years without a clean resolution.

The gap between GPL and MIT comes down to a genuine philosophical difference, not just a legal technicality. MIT is permissive: use it, modify it, ship it in a proprietary product, no strings attached. GPL is copyleft: anything you distribute that's derived from the code has to stay open under the same terms. MIT spreads adoption by getting out of the way. GPL spreads freedom by making sure nobody can take from the pool without putting back.

If you're picking a license for something you built, the question is really about what you want. A library you mostly want people to actually use? MIT is probably right. A foundational project where you don't want a company forking it and locking the improvements away from everyone who made it worth forking in the first place? That's GPL. That's exactly why Torvalds chose it.

Stallman's "free as in free beer versus free as in freedom" line gets at something real. Early Unix from companies like SCO cost serious money. Linux distributions can also cost money, Red Hat Enterprise Linux runs thousands of dollars a year in support subscriptions, but every bit of it is open. You can read it, modify it, redistribute it. That's free as in freedom. Plenty of apps that cost nothing give you none of that. Free to download, completely locked down. The price and the freedom are just separate questions.

A3. The Ethics of Open Source

Whether all software should be open source is a genuinely hard question, and anyone who acts like it isn't probably hasn't thought about it seriously.
The argument for universal open source is strong. Software is infrastructure now. Billions of people's communications, medical records, financial transactions, and political lives run through systems they can't see into, and that's a real democratic problem. You can't meaningfully consent to something you're not allowed to inspect. Open source allows independent security audits, keeps communities from being held hostage by vendors, and lets people maintain software long after the company behind it loses interest. The COVID-19 pandemic made this concrete: open-source epidemiological models and genomic tools got picked apart and improved by scientists all over the world in ways proprietary equivalents simply couldn't match.
But the case against isn't just corporate lobbying dressed up as philosophy. Some software genuinely needs sustained commercial investment that volunteer communities can't provide. Proprietary development can move faster in certain contexts and fund research that open communities rarely touch. Security through obscurity is mostly a myth, but there's a legitimate version of the argument that publishing source code for critical infrastructure does make targeted attacks cheaper. And the economic model of open source has a quiet, uncomfortable problem: a small number of people maintain critical internet infrastructure as a side project, often unpaid, and when they burn out, everyone pays for it. The Heartbleed vulnerability in

OpenSSL, maintained by a tiny underfunded team, broke a huge chunk of the internet's security. That wasn't a fluke. It was a structural failure.

The corporate extraction problem is where it gets genuinely uncomfortable. Google's Android runs on the Linux kernel. Amazon's AWS runs on Linux. Meta's entire stack is built on open-source components. These companies make hundreds of billions of dollars on foundations laid by volunteers and academics. They do contribute back, Google employs kernel developers, Meta has open-sourced real tools, but it's not proportionate. Not even close. Whether that's exploitation or just the predictable outcome of releasing software under open terms is something the community keeps arguing about without resolution. Red Hat's 2023 decision to restrict access to RHEL source code poured fuel on that argument, because it felt like exactly the kind of enclosure the GPL was designed to prevent.

Newton's "standing on the shoulders of giants" maps onto this almost too well. Every serious piece of software written today builds on decades of prior work: operating systems, compilers, networking protocols, algorithms, most of it developed in universities and research labs and released openly. Open source doesn't slow down innovation. It speeds it up by removing the need to rebuild things that already exist. The real question is whether the current setup, where companies can extract enormous value without giving back proportionately, will eventually burn out the people holding it all together.

3. Part B : Linux Footprint on a System

Installation Method

The Linux Kernel is not installed as an package, it is present from the moment a distribution is installed. On Debian/Ubuntu systems, the kernel is managed via the APT package manager. The current kernel package is named linux-image-$(uname -r). A new kernel version can be installed with:

```
sudo apt install linux-image-6.x.x-xx-generic
```

On RPM-based systems such as Fedora and CentOS, the equivalent is sudo dnf update kernel. Compiling from source downloading tarball from kernel.org, configuring with make menuconfig, and running `make && make module_install && make install` is also fully supported and is the method used by developers and those needing custom configurations. The kernel.org website distributes official release tarballs, and Linus Torvalds' own git tree i publicly accessible.

Key Directories

Directory	Purpose
/boot/vmlinuz-*	The compressed kernel image itself, loaded by the bootloader (GRUB)
/boot/initrd.img-*	Initial RAM disk a temporary root filesystem used during early boot
/boot/System.map-*	Symbol table mapping kernel function names to memory addresses
/lib/modules/$(uname -r)/	Loadable kernel modules (.ko files) for drivers and filesystems
/proc/	Virtual filesystem exposing live kernel state, processes, memory, hardware info
/sys/	sysfs : kernel's view of hardware devices and drivers, readable and writable
/etc/modprobe.d/	Configuration files for module loading behavior
/etc/sysctl.conf	Kernel parameter tuning : network, memory, security settings

User and Group: Security Implications

The kernel doesn't run like a normal process. It's not sitting under some user account doing its thing alongside everything else. It runs in ring 0, kernel mode, the highest privilege level the CPU has, with direct unrestricted access to hardware. Every application, every shell, every service you run lives in ring 3, userspace, where the CPU enforces hard limits on what the software can touch. That separation is the whole foundation of Linux security. A bug in a userspace program crashes that program, in the kernel can compromise the entire machine.

Kernel modules, the drivers and extensions loaded at runtime, run in kernel mode too. A malicious module, , has the same unrestricted hardware access as the kernel itself. Modern distributions handle this with Secure Boot and module signing, making sure only verified modules get loaded in the first place. Root can still load modules manually with insmod or modprobe.

Service Management and Kernel Commands

Task	Command
Check current kernel version	uname -r
List loaded kernel modules	lsmod
Load a module	sudo modprobe <module_name>
Remove a module	sudo modprobe -r <module_name>
View kernel ring buffer (boot log)	dmesg | less
View/set kernel parameters	sysctl -a / sudo sysctl -w param=value
Update GRUB after new kernel install	sudo update-grub
Boot into specific kernel (edit GRUB)	/etc/default/grub → GRUB_DEFAULT

Update Model : Security Patches

Linus Torvalds manages the mainline kernel tree and cuts a new major version, something like 6.8 or 6.9, roughly every 9 to 10 weeks. Greg Kroah-Hartman handles the Long Term Support branches, which get security and critical bug fixes for up to six years. kernel.org keeps a current list of which LTS branches are still active. Distributions like Ubuntu LTS and Debian Stable pick a specific LTS branch and backport security fixes to it, so you're not forced to track the bleeding edge just to stay secure.

When a vulnerability is found, it goes to the kernel security team through a private disclosure process, then the fix lands in the mainline tree, flows down to the LTS branches, and eventually hits distribution package repositories. On a Debian or Ubuntu system, `sudo apt upgrade` handles it. Nothing exotic.

The part that's easy to overlook is the transparency. Every single patch is visible in the kernel's git history, with commit messages explaining what broke and how it was fixed. You can audit the entire chain. That's not something you get with a proprietary OS, where security updates arrive as opaque binaries and you're just expected to trust the vendor. With Linux, you don't have to take anyone's word for it.

4. Part C : The FOSS Ecosystem

Dependencies : What the Kernel Builds On

The Linux Kernel itself has only few runtime dependencies : by design. As the lowest layer of the software stack, it cannot depend on higher-level libraries. However, building and working with the kernel depends on several critical tools: the GNU C Compiler (GCC) or Clang/LLVM for compilation, GNU Make for the build system, GNU Binutils for linking and assembling, libelf for working with ELF binaries during module loading, OpenSSL for generating module signing keys, and Flex/Bison for parsing kernel configuration. The tight relationship between the kernel and the GNU toolchain is why the complete system i properly called GNU/Linux : both halves are essential.

What the Kernel Has Enabled : Forks, Derivatives, and the Stack

The Linux Kernel's ecosystem is staggering in scope. Android: running on over 3 billion active devices : is built on a modified Linux kernel. The Chrome OS kernel is Linux. Every major cloud provider (AWS, GCP, Azure) runs Linux on its hypervisors and virtual machines. The Steam Deck runs Linux. The International Space Station's laptops run Linux. The world's top 500 supercomputers all run Linux. This ubiquity is not coincidence : it is the direct consequence of the GPL license, which guaranteed that improvements to the kernel could not be captured and privatized.

In the LAMP stack (Linux, Apache, MySQL, PHP/Python/Perl), Linux is the foundation layer. It provides process isolation, the TCP/IP networking stack, the filesystem abstraction, and the security model that makes web serving possible. Apache, MySQL, and the scripting languages run in userspace and depend on the kernel's system calls : read(), write(), socket(), fork(), exec() : for everything they do. Remove the kernel and the entire stack collapses.

Notable forks and derivatives include: the Realtime Linux (PREEMPT_RT) patch set, used in industrial control systems and audio production; the gsecurity hardening patches; and the Linux-libre project, which removes non-free firmware blobs from the kernel tree for distributions like Trisquel. Android's kernel modifications : Binder IPC, ION memory allocator, paranoid network security: represent the largest and most commercially significant fork,though Google has been progressively upstreaming many of these features.

Community and Governance

The Linux Kernel is governed through a hierarchical maintainer model. Linus Torvalds holds final authority over the mainline tree. Each subsystem (networking, filesystems, drivers, security, etc.) has one or more designated maintainers who review and accept patches for their area. The MAINTAINERS file in the kernel source tree : over 24,000 lines long : documents this structure explicitly. Development happens on the Linux Kernel Mailing List (LKML), one of the highest-volume technical mailing lists in existence, and on subsystem-specific lists. The kernel community does not use GitHub pull requests; patches are submitted as plain text emails, reviewed publicly, and accepted or rejected with detailed technical feedback.

The Linux Foundation, founded in 2000, provides organizational and financial support. It employs Torvalds and several key maintainers, funds infrastructure, and organizes the Linux

Plumbers Conference and Linux Foundation Summit. Major corporate contributors include Intel, Google, Red Hat, Samsung, NVIDIA, and Huawei, all of whom employ engineers who contribute kernel patches as their primary job function. This corporate involvement is both the kernel's strength : it provides sustainable funding and engineering resources : and a source of ongoing tension about the direction of development.

5. Part D : Open Source vs Proprietary: Critical Analysis

The closest proprietary alternative to the Linux Kernel is the Windows NT Kernel (the kernel underlying all modern Windows versions) and Apple's XNU Kernel (used in macOS, iOS, and all Apple platforms). The comparison below primarily focuses on Windows NT as the more direct competitor in the server and infrastructure space.

Dimension	Linux Kernel (Open Source)	Windows NT Kernel (Proprietary)
Cost	Free of charge. No licensing fees for the kernel itself. Distributions may charge for support (RHEL: $1,400/server/year).	Bundled with Windows Server licenses. Windows Server 2022 Standard costs approx.. $1,069 per license.
Security Auditing	Full source code publicly available. Anyone : security researchers, governments, academics : can audit every line.	Source code is proprietary. Microsoft conducts internal audits and shares limited access under NDA.
Support & Reliability	Enterprise support via Red Hat, SUSE, Canonical. SLAs are commercially available. Community support is vast.	Comprehensive Microsoft support with clear SLAs. Extended Security Updates available.
Freedom to Modify	Complete freedom. Any organization can modify the kernel for their hardware. ARM SoC vendors, cloud providers all do this.	No source access for commercial users. Hardware vendors must work within Microsoft's constraints.
Community vs Corporate Control	Governance is distributed but Torvalds retains final authority. Corporate contributors fund development but cannot dictate direction.	Entirely under Microsoft's control. Microsoft can change direction, deprecate features, or end support.
Overal Verdict	For server infrastructure, cloud, embedded, and any deployment where long-term independence, auditability, and cost matter: Linux wins.	Remains relevant in enterprise desktop environments, Active Directory integration, and Microsoft-centric ecosystems.

For server, cloud, and embedded infrastructure, the call is easy: run Linux. Zero licensing cost, full source transparency, enterprise support from multiple vendors competing for your business, three decades of proven stability, and the ability to actually read and modify the system when something goes wrong. Linux holds over 96% of the web server market not because of ideology but because thousands of organizations evaluated their options and kept landing on the same answer.

Contributing to it is worth doing. No other codebase a systems programmer can work on has this kind of reach. Even a small fix, a driver patch, a documentation correction, a missing test

case, can end up running on billions of devices. The process is hard. Patches get reviewed carefully, the coding standards don't bend, and the mailing list culture can be blunt to the point of being off-putting. But that rigor isn't bureaucracy for its own sake. The stakes are just genuinely high, and the process reflects that. It's worth it.

6. Conclusion

At its core, Linux is a story about what happens when someone solves a personal problem, shares the solution openly, and uses a license that stops anyone from locking it down. Torvalds wasn't trying to change computing. He wanted a free, modifiable system for his own machine. The GPL and the culture of the early internet did the rest. Vy requiring that every improvement stay free, the GPL turned a hobby project into the infrastructure the global digital economy runs on.

This audit has looked at that story from several angles: the historical frustration that started it, the legal text that governs it, the ethical tensions it still hasn't fully resolved, its practical footprint on a running system, the ecosystem it's enabled, and an honest look at how it stacks up against proprietary alternatives. What comes out of that isn't just "Linux is technically good," though it is. It's that the development model itself means something. It's proof that complex, critical infrastructure can be built by a distributed community, owned by no single company, readable by anyone, and governed by the quality of your contributions rather than your budget.

The problems are real and worth taking seriously. Maintainer burnout, lopsided corporate extraction, governance friction, a codebase pushing past 35 million lines as of 2024. These aren't hypothetical risks. But Linux has been absorbing that kind of pressure for 33 years and keeps getting more capable and more widely deployed. Understanding why it was built, how it's licensed, and what it actually means to share knowledge openly isn't background reading. It's the foundation.

References

[1] Torvalds, L. (1991). Initial announcement, comp.os.minix newsgroup, August 25, 1991.

[2] Stallman, R. (1983). Initial GNU announcement. gnu.org/gnu/initial-announcement.html

[3] Free Software Foundation. The Free Software Definition. gnu.org/philosophy/free-sw.html

[4] Open Source Initiative. The Open Source Definition. opensource.org/osd

[5] Raymond, E. S. (1999). The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar

[6] Torvalds, L. & Diamond, D. (2001). Just for Fun: The Story of an Accidental Revolutionary.

[7] The Linux Kernel documentation. kernel.org/doc/html/latest/

[8] SPDX License List : GPL-2.0-only. spdx.org/licenses/GPL-2.0-only.html

[9] Linux Foundation. (2023). Linux Kernel Development Report. linuxfoundation.org

[10] Kroah-Hartman, G. (2024). Linux Kernel LTS releases. kernel.org